

EFREI Paris — Databases 2 (TI603I)

Noah Delhi

Beya Heni

Christine Khazaka

ArtConnect Pro

Final Project Report

Project Summary

Design and implementation of a relational MySQL database integrated into a JavaFX application. The system manages artists, artworks, galleries, exhibitions, workshops, and community members.

Tech Stack

Java 23 • MySQL 8 • JDBC • Maven

Academic Year 2025–2026

Contents

1	Project Introduction	3
1.1	Context and Objective	3
1.2	Project Structure	3
1.3	Deliverables	4
2	Step 1 — Functional Analysis	5
2.1	Application Features	5
2.2	User Roles	5
2.3	Use Case Diagram	7
2.4	Class Diagrams	8
3	Step 2 — Database Design	10
3.1	Identified Entities and Attributes	10
3.2	Conceptual Data Model (CDM / ERD)	11
3.3	Logical Data Model (LDM)	11
3.4	Normalization (up to 3NF)	12
3.4.1	First Normal Form (1NF)	12
3.4.2	Second Normal Form (2NF)	12
3.4.3	Third Normal Form (3NF)	12
3.4.4	Key Constraints	12
3.5	SQL Schema (excerpt)	12
4	Step 3 — Advanced SQL Features	14
4.1	Sample Data	14
4.2	Views	14
4.3	Indexes	15
4.4	Triggers	16
4.5	Stored Procedures and Functions	17
4.6	Transactions	18
4.6.1	Scenario 1 — Successful Registration (COMMIT)	18
4.6.2	Scenario 2 — Failed Registration (ROLLBACK)	18
5	Step 4 — JDBC Integration	20
5.1	Application Architecture	20
5.2	Entity Classes (Model Layer)	21
5.3	DAO Interfaces	21
5.4	JDBC DAO Implementations	21
5.5	Connection Pool — HikariCP	22

5.6	Service Layer and Dependency Injection	23
5.7	Application Screenshots	23
6	Step 5 — Finalization and Testing	25
6.1	Test Scenarios	25
6.2	Challenges and Solutions	26
7	Conclusion	27

Chapter 1

Project Introduction

1.1 Context and Objective

This project was completed as part of the *Databases 2* course (TI603I) at EFREI Paris. The objective was to design and implement a relational database from scratch, then integrate it into an existing Java application skeleton called **ArtConnect Pro**.

ArtConnect Pro is a community platform that aims to promote the local art scene. It allows users to:

- Connect artists (painters, musicians, photographers, etc.)
- Showcase their artworks
- Organize events (exhibitions, concerts, workshops)
- Allow community members to discover artists, follow their events, and register for them

The provided Java application worked exclusively with in-memory data (pre-filled Java lists) and InMemory service implementations. Our task was to replace this with a fully functional MySQL persistence layer.

1.2 Project Structure

The project was organized into 5 steps over 6 supervised sessions:

Step	Description	Status
1	Functional analysis — UML diagrams, feature identification	Complete
2	Conceptual & Logical modeling — CDM/LDM, 3NF normalization	Complete
3	DB implementation — views, indexes, triggers, procedures, transactions	Complete
4	JDBC integration — DAOs, services, UI connected to MySQL	Complete
5	Finalization — tests, refinements, documentation	Complete

Table 1.1: Project steps and completion status

1.3 Deliverables

- This final report (CDM/LDM diagrams, SQL scripts, code excerpts, test scenarios)
- GitHub repository with complete project code and SQL scripts
- Application screenshots demonstrating live database integration
- Presentation slides

Chapter 2

Step 1 — Functional Analysis

2.1 Application Features

The application is organized into **7 tabs**, each corresponding to a domain:

Tab	Features (from user perspective)
Artists	List all artists · Search by name · Filter by discipline · Filter by city · View details
Artworks	List all artworks · Filter by artist / type / status · View details (title, medium, price)
Exhibitions	List all exhibitions · Filter by gallery / date range · View artworks and curator
Galleries	List all galleries · View rating and contact info · Browse hosted exhibitions
Workshops	List all workshops · Filter by level / instructor · View date, location, price
Community	List members · View bookings and reviews · Filter by city / membership type
Discover	Featured exhibitions (cards) · Upcoming workshops (cards)

Table 2.1: Application features by tab

Planned features (not yet in current interface): user authentication, workshop booking button, review submission, artist profile creation.

2.2 User Roles

Organizer (Artist) Creates and manages their profile, adds/edits/deletes artworks, proposes workshops as instructor, participates in exhibitions.

Community Member Browses artists/artworks/events, books workshops, writes reviews on artworks (rating 1–5), has favorite disciplines and a membership type (free/premium).

2.3 Use Case Diagram

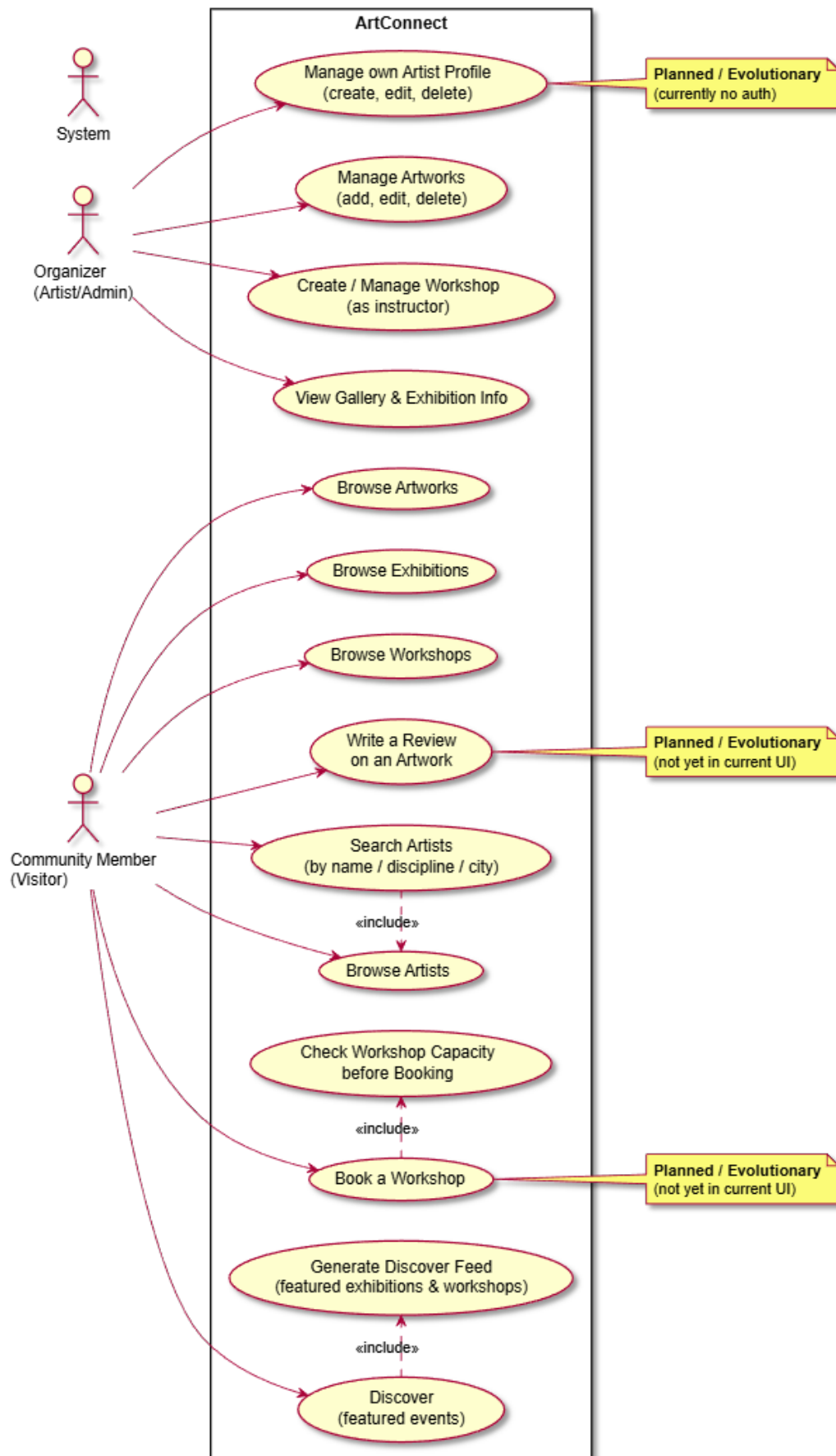


Figure 2.1: UML Use Case Diagram — ArtConnect Pro

2.4 Class Diagrams

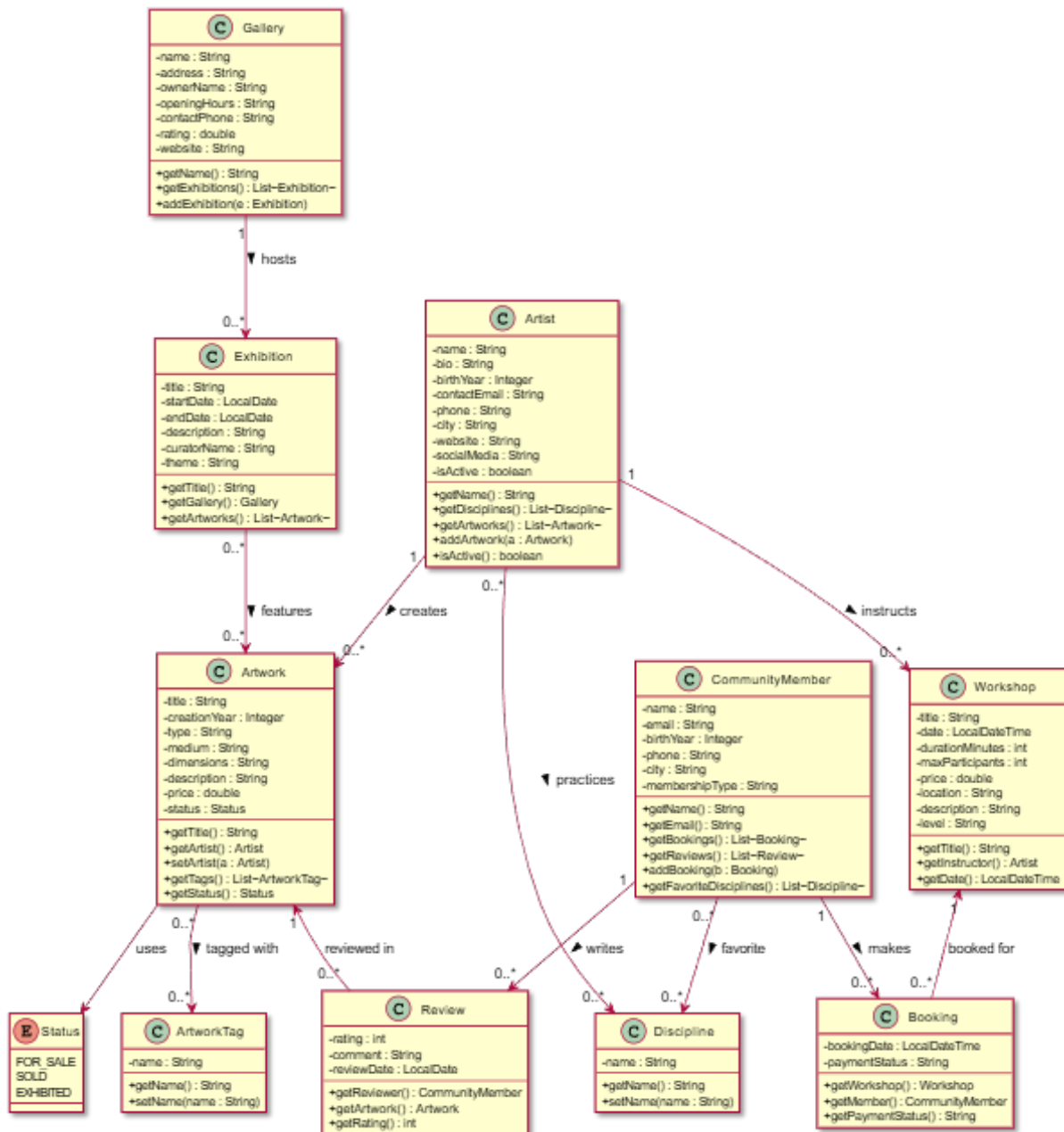


Figure 2.2: Domain Model — Java entity classes

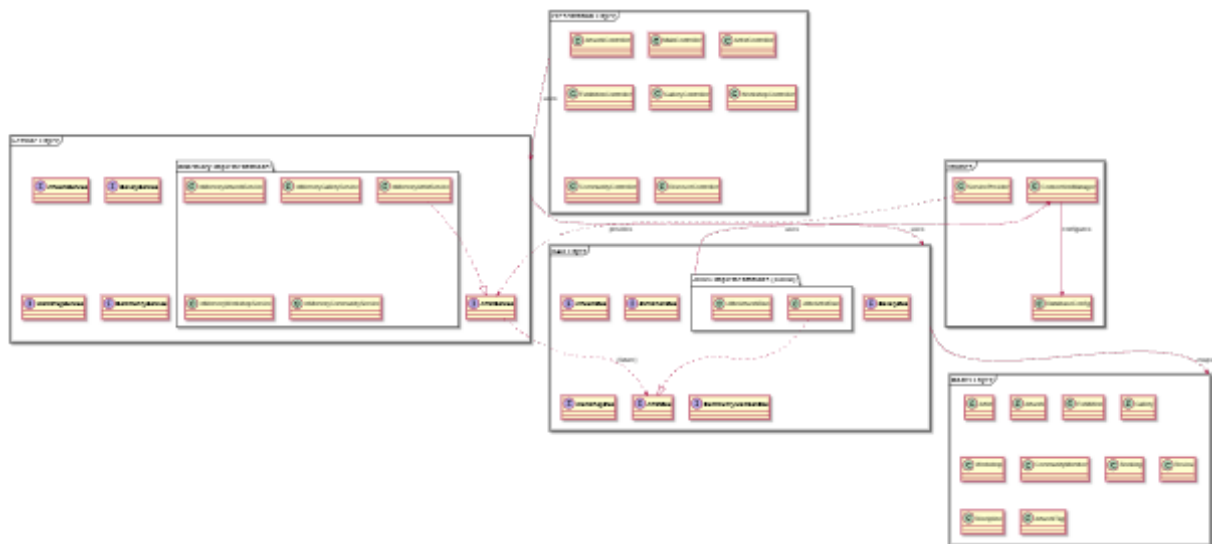


Figure 2.3: Application architecture — layered diagram

Chapter 3

Step 2 — Database Design

3.1 Identified Entities and Attributes

Based on the Java model classes and business requirements, 10 entities were identified:

Entity	Key Attributes	Identifier
Artist	name, bio, birth_year, contact_email, phone, city, website, is_active	artist_id (PK)
Artwork	title, creation_year, type, medium, price, status	artwork_id (PK)
Gallery	name, address, owner_name, opening_hours, rating, website	gallery_id (PK)
Exhibition	title, start_date, end_date, description, curator_name, theme	exhibition_id (PK)
Workshop	title, date, duration_minutes, max_participants, price, level	workshop_id (PK)
CommunityMember	name, email, birth_year, city, membership_type	member_id (PK)
Booking	booking_date, payment_status	booking_id (PK)
Review	rating (1–5), comment, review_date	review_id (PK)
Discipline	name	discipline_id (PK)
ArtworkTag	name	tag_id (PK)

Table 3.1: Identified entities and identifiers

3.2 Conceptual Data Model (CDM / ERD)

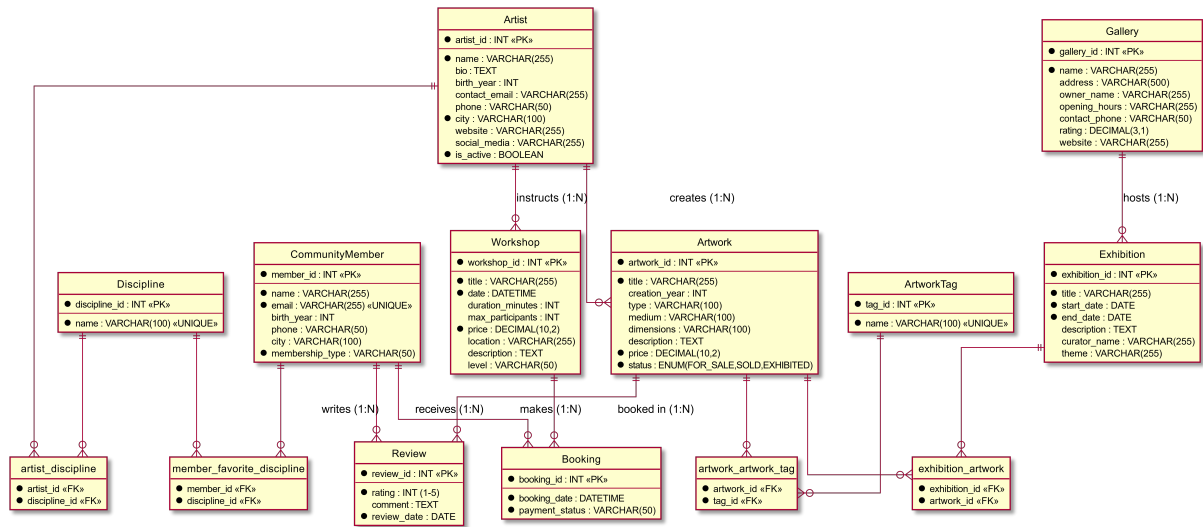


Figure 3.1: Conceptual Data Model — Entity-Relationship Diagram

3.3 Logical Data Model (LDM)

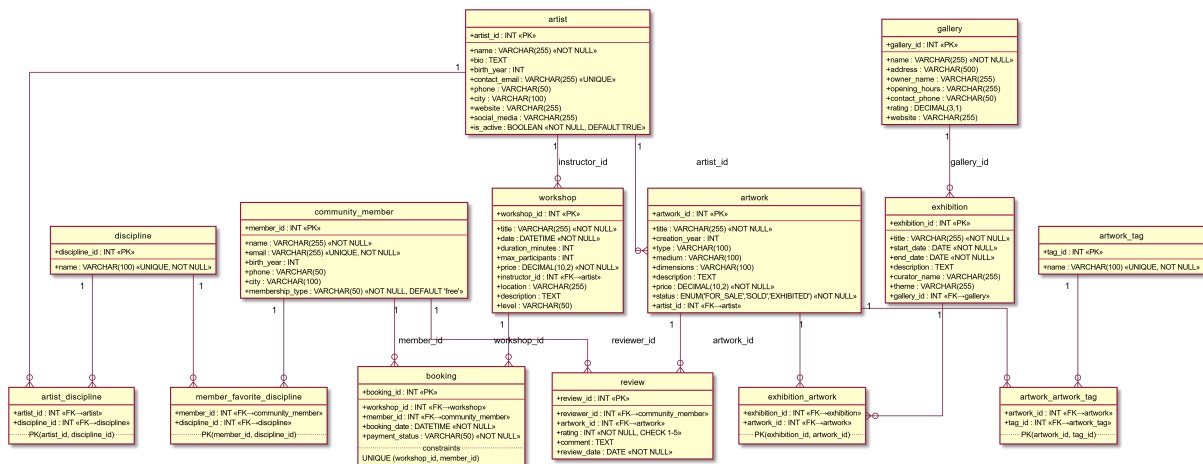


Figure 3.2: Logical Data Model — Tables, keys and relationships

The database consists of **14 tables** in total: 10 core tables and 4 junction tables for Many-to-Many relationships:

Junction Table

artist_discipline

artwork_artwork_tag

exhibition_artwork

member_favorite_discipline

Relationship

Artist ↔ Discipline (N:M)

Artwork ↔ ArtworkTag (N:M)

Exhibition ↔ Artwork (N:M)

CommunityMember ↔ Discipline (N:M)

Table 3.2: Junction tables for Many-to-Many relationships

3.4 Normalization (up to 3NF)

3.4.1 First Normal Form (1NF)

Rule: Each column must contain atomic values; no repeating groups.

Applied: In the Java model, entities use `List<Discipline>`, `List<ArtworkTag>`, etc. These multi-valued attributes were extracted into dedicated junction tables rather than stored as comma-separated values. All tables contain only atomic column values.

3.4.2 Second Normal Form (2NF)

Rule: Every non-key attribute must depend on the *whole* primary key (relevant for composite PKs).

Applied: All core tables use a single-column surrogate key (`INT AUTO_INCREMENT`). Junction tables have composite PKs but carry no non-key attributes (except `booking` which received a surrogate `booking_id` with a `UNIQUE` constraint on the pair (`workshop_id`, `member_id`)).

3.4.3 Third Normal Form (3NF)

Rule: No non-key attribute may transitively depend on the primary key.

Applied and verified: No transitive dependencies exist in any table. For example, `artwork.artist_id` is a foreign key — the artist's name and city are stored in the `artist` table, not repeated in `artwork`. Similarly, `exhibition.gallery_id` is a FK; the gallery's address is not duplicated in the `exhibition` table.

3.4.4 Key Constraints

- `artist.contact_email` — `UNIQUE`
- `community_member.email` — `UNIQUE`
- `discipline.name`, `artwork_tag.name` — `UNIQUE`
- `booking(workshop_id, member_id)` — `UNIQUE` (no double booking)
- `exhibition.end_date >= start_date` — `CHECK`
- `review.rating BETWEEN 1 AND 5` — `CHECK`
- `membership_type IN ('free', 'premium')` — `CHECK`

3.5 SQL Schema (excerpt)

```

1 CREATE TABLE artist (
2     artist_id      INT AUTO_INCREMENT PRIMARY KEY,
3     name           VARCHAR(255) NOT NULL,
4     bio            TEXT,
5     birth_year     INT,
6     contact_email  VARCHAR(255) UNIQUE,
7     phone          VARCHAR(50),

```

```
8      city          VARCHAR(100),
9      website       VARCHAR(255),
10     social_media   VARCHAR(255),
11     is_active      BOOLEAN NOT NULL DEFAULT TRUE
12 );
13
14 CREATE TABLE artwork (
15     artwork_id     INT AUTO_INCREMENT PRIMARY KEY,
16     title           VARCHAR(255) NOT NULL,
17     creation_year   INT,
18     type            VARCHAR(100),
19     medium          VARCHAR(100),
20     price           DECIMAL(10,2) NOT NULL,
21     status          ENUM('FOR_SALE', 'SOLD', 'EXHIBITED') NOT NULL DEFAULT '
FOR_SALE',
22     artist_id      INT,
23     CONSTRAINT fk_artwork_artist
24         FOREIGN KEY (artist_id) REFERENCES artist(artist_id)
25         ON DELETE CASCADE ON UPDATE CASCADE
26 );
27
28 CREATE TABLE exhibition (
29     exhibition_id   INT AUTO_INCREMENT PRIMARY KEY,
30     title           VARCHAR(255) NOT NULL,
31     start_date      DATE NOT NULL,
32     end_date        DATE NOT NULL,
33     description     TEXT,
34     curator_name    VARCHAR(255),
35     theme           VARCHAR(255),
36     gallery_id      INT,
37     CONSTRAINT chk_dates CHECK (end_date >= start_date),
38     CONSTRAINT fk_exhibition_gallery
39         FOREIGN KEY (gallery_id) REFERENCES gallery(gallery_id)
40         ON DELETE SET NULL ON UPDATE CASCADE
41 );
```

Listing 3.1: Schema excerpt — artist and artwork tables

Chapter 4

Step 3 — Advanced SQL Features

4.1 Sample Data

Sample data was generated using an AI assistant (Claude) with a detailed prompt specifying realistic data across all entities. The data was verified and adjusted manually before use.

Entity	Count	Notes
discipline	5	Painting, Sculpture, Photography, Digital Art, Mixed Media
artist	6	From Paris, Rome, Tokyo, Dakar, Stockholm, Brussels
artwork	15	8 FOR_SALE, 4 SOLD, 3 EXHIBITED
gallery	4	Paris, Rome, Stockholm, London
exhibition	5	Date ranges spanning 2024–2025
workshop	6	2 Beginner, 2 Intermediate, 2 Advanced
community_member	8	4 free, 4 premium
booking	12	8 PAID, 2 PENDING, 2 CANCELLED
review	10	Ratings from 3 to 5 stars

Table 4.1: Sample data counts

4.2 Views

5 views were created, each with a clear objective:

View	Objective	Description
v_artist_portfolio	Query simplification	Artist info + disciplines (GROUP_CONCAT) + artwork count + avg price
v_exhibition_details	Query simplification	Exhibition + gallery info + artwork count + duration in days
v_workshop_availability	Complexity hiding	Available spots = max_participants - active bookings
v_member_activity	Security	Member summary without sensitive data (email/phone hidden)
v_artwork_catalog	Query simplification	Artwork + artist + tags + avg rating for frontend display

Table 4.2: Views and their objectives

```

1 CREATE OR REPLACE VIEW v_workshop_availability AS
2 SELECT
3     w.workshop_id,
4     w.title           AS workshop_title,
5     w.date,
6     a.name            AS instructor_name,
7     w.level,
8     w.price,
9     w.max_participants,
10    COUNT(CASE WHEN b.payment_status IN ('PAID','PENDING') THEN 1 END)
11          AS confirmed_bookings,
12    w.max_participants -
13    COUNT(CASE WHEN b.payment_status IN ('PAID','PENDING') THEN 1 END)
14          AS available_spots
15 FROM workshop w
16 JOIN artist a ON w.instructor_id = a.artist_id
17 LEFT JOIN booking b ON w.workshop_id = b.workshop_id
18 GROUP BY w.workshop_id, w.title, w.date, a.name, w.level,
19          w.price, w.max_participants;

```

Listing 4.1: View: v_workshop_availability

4.3 Indexes

6 indexes were created to optimize the most frequent queries:

Index	Table.Column(s)	Justification
idx_artwork_artist	artwork(artist_id)	Frequent JOIN between artwork and artist
idx_artwork_status	artwork(status)	Frequent WHERE status = 'FOR_SALE'
idx_exhibition_dates	exhibition(start_date, end_date)	Composite index for date range queries
idx_review_artwork	review(artwork_id)	AVG(rating) aggregations in catalog view
idx_booking_workshop	booking(workshop_id)	Booking count per workshop (capacity trigger)
idx_workshop_date	workshop(date)	Sort and filter workshops by date

Table 4.3: Indexes and their justification

4.4 Triggers

4 triggers were implemented for business rule enforcement and auditing:

Trigger	Event	Purpose
trg_check_workshop_capacity	BEFORE INSERT ON booking	Blocks overbooking when PAID + PENDING $\geq$ max_participants
trg_check_exhibition_date	BEFORE UPDATE ON exhibition	Blocks updates where end_date < start_date
trg_audit_artwork_status	AFTER UPDATE ON artwork	Logs every status change to audit_log table
trg_set_artwork_exhibited	AFTER INSERT ON exhibition_artwork	Auto-sets artwork status to EXHIBITED if exhibition is currently active

Table 4.4: Triggers and their purposes

```

1 DELIMITER $$
2 CREATE TRIGGER trg_check_workshop_capacity
3 BEFORE INSERT ON booking
4 FOR EACH ROW
5 BEGIN
6     DECLARE active_bookings INT;
7     DECLARE max_cap INT;
8
9     SELECT COUNT(*) INTO active_bookings
10    FROM booking
11   WHERE workshop_id = NEW.workshop_id
12         AND payment_status IN ('PAID', 'PENDING');
13

```

```

14 SELECT max_participants INTO max_cap
15 FROM workshop WHERE workshop_id = NEW.workshop_id;
16
17 IF active_bookings >= max_cap THEN
18     SIGNAL SQLSTATE '45000'
19     SET MESSAGE_TEXT = 'Workshop is full: no more spots
available.';
20 END IF;
21 END$$
22 DELIMITER ;

```

Listing 4.2: Trigger: trg_check_workshop_capacity

```

1 DELIMITER $$
2 CREATE TRIGGER trg_audit_artwork_status
3 AFTER UPDATE ON artwork
4 FOR EACH ROW
5 BEGIN
6     IF OLD.status <> NEW.status THEN
7         INSERT INTO audit_log
8         (table_name, record_id, action, old_value, new_value,
changed_at)
9         VALUES
10        ('artwork', NEW.artwork_id, 'STATUS_CHANGE',
OLD.status, NEW.status, NOW());
11     END IF;
12 END$$
13 DELIMITER ;

```

Listing 4.3: Trigger: trg_audit_artwork_status

4.5 Stored Procedures and Functions

5 stored programs were created:

Object	Type	Purpose
fn_get_workshop_available_spots(<i>id</i>)	FUNCTION	Returns remaining spots for a workshop
fn_get_artist_average_rating(<i>id</i>)	FUNCTION	Returns average rating across all artist artworks
sp_create_workshop_with_artist(<i>id</i>)	PROCEDURE	Creates workshop + auto-creates artist if not found by email
sp_register_member_to_workshop(<i>ids</i>)	PROCEDURE	Batch-registers a member, skips full or already-booked
sp_generate_artist_report(<i>id</i>)	PROCEDURE	Returns 4 result sets: info, art-works, exhibitions, workshops

Table 4.5: Stored procedures and functions

```

1 DELIMITER $$
2 CREATE FUNCTION fn_get_workshop_available_spots(p_workshop_id INT)
3 RETURNS INT

```

```

4 DETERMINISTIC
5 READS SQL DATA
6 BEGIN
7     DECLARE total_cap INT;
8     DECLARE active_bookings INT;
9
10    SELECT max_participants INTO total_cap
11    FROM workshop WHERE workshop_id = p_workshop_id;
12
13    SELECT COUNT(*) INTO active_bookings
14    FROM booking
15    WHERE workshop_id = p_workshop_id
16        AND payment_status IN ('PAID', 'PENDING');
17
18    RETURN total_cap - active_bookings;
19 END$$
20 DELIMITER ;

```

Listing 4.4: Function: fn_get_workshop_available_spots

4.6 Transactions

Two transactional scenarios were implemented to demonstrate atomicity:

4.6.1 Scenario 1 — Successful Registration (COMMIT)

A new premium member is created atomically:

1. Insert into `community_member`
2. Insert 2 favorite disciplines into `member_favorite_discipline`
3. Book 3 workshops (3 inserts into `booking`)
4. Write 2 reviews (2 inserts into `review`)

All operations succeed → COMMIT. The member exists with all her data.

4.6.2 Scenario 2 — Failed Registration (ROLLBACK)

A member attempts to register and book a workshop that is full. The `trg_check_workshop_capacity` trigger fires a `SIGNAL SQLSTATE '45000'` error, caught by `DECLARE EXIT HANDLER FOR SQLEXCEPTION` → full ROLLBACK. The member is never created in the database.

```

1 DELIMITER $$
2 CREATE PROCEDURE sp_demo_atomic_registration()
3 BEGIN
4     DECLARE EXIT HANDLER FOR SQLEXCEPTION
5     BEGIN
6         ROLLBACK;
7         SELECT 'Transaction rolled back: an error occurred.' AS result;
8     END;
9
10    START TRANSACTION;
11
12    INSERT INTO community_member (name, email, membership_type)

```

```
13  VALUES ('Marc Lef vre', 'marc.lefevre@test.com', 'free');
14
15  -- This will fail if the workshop is full (trigger fires)
16  INSERT INTO booking (workshop_id, member_id, payment_status)
17  VALUES (2, LAST_INSERT_ID(), 'PENDING');
18
19  COMMIT;
20  SELECT 'Registration successful.' AS result;
21 END$$
22 DELIMITER ;
```

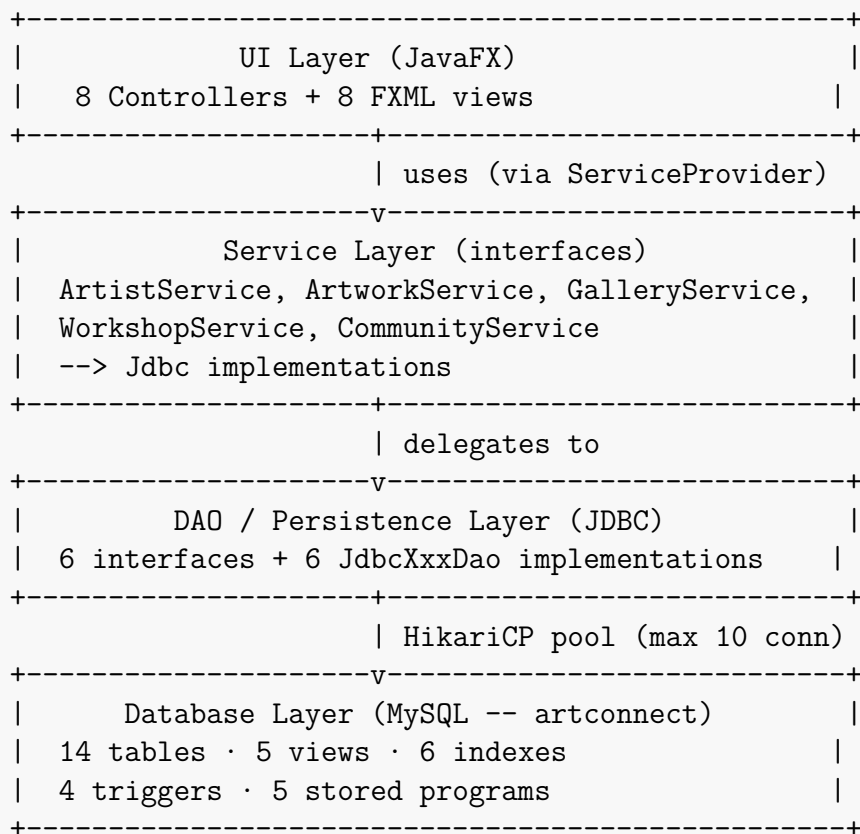
Listing 4.5: Transaction with ROLLBACK handler

Chapter 5

Step 4 — JDBC Integration

5.1 Application Architecture

The application follows a **3-tier layered architecture**:



5.2 Entity Classes (Model Layer)

Entity	Key Fields
Artist	name, bio, birthYear, disciplines, contactEmail, city, isActive
Artwork	title, creationYear, type, medium, price, status (enum), artist
Exhibition	title, startDate, endDate, gallery, curatorName, artworks
Gallery	name, address, ownerName, rating, exhibitions
Workshop	title, date, durationMinutes, maxParticipants, price, instructor
CommunityMember	name, email, birthYear, city, membershipType, bookings, reviews
Booking	workshop, member, bookingDate, paymentStatus
Review	reviewer, artwork, rating, comment, reviewDate
Discipline	name
ArtworkTag	name

Table 5.1: Entity classes and key fields

Design note: Java entity classes have **no id fields** — they use direct object references. Database IDs exist only at the SQL level (surrogate `INT AUTO_INCREMENT`). DAOs resolve IDs via unique fields (name, email) when writing.

5.3 DAO Interfaces

6 DAO interfaces were defined in the `com.project.artconnect.dao` package, all providing full CRUD operations:

```

1 public interface ArtistDao {
2     List<Artist> findAll();
3     List<Artist> findByCity(String city);
4     void save(Artist artist);
5     void update(Artist artist);
6     void delete(String artistName);
7 }

```

Listing 5.1: ArtistDao interface

5.4 JDBC DAO Implementations

All 6 JDBC DAO implementations share the following patterns:

- **PreparedStatement** — used for all SQL to prevent SQL injection
- **try-with-resources** — guarantees connection/statement closing
- **Transactions** — `setAutoCommit(false)`, `commit()`, `rollback()` for multi-table writes

- **Batch operations** — `addBatch()` / `executeBatch()` for junction table inserts
- **Generated keys** — `RETURN_GENERATED_KEYS` for auto-increment IDs

```

1 @Override
2 public void save(Artist artist) {
3     String insertArtist =
4         "INSERT INTO artist (name, bio, birth_year, contact_email, " +
5         "phone, city, website, social_media, is_active) " +
6         "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
7
8     try (Connection conn = ConnectionManager.getConnection()) {
9         conn.setAutoCommit(false);
10        try {
11            int artistId;
12            try (PreparedStatement ps = conn.prepareStatement(
13                insertArtist, Statement.RETURN_GENERATED_KEYS)) {
14                ps.setString(1, artist.getName());
15                ps.setString(2, artist.getBio());
16                ps.setObject(3, artist.getBirthYear(), Types.INTEGER);
17                ps.setString(4, artist.getContactEmail());
18                // ... remaining fields
19                ps.executeUpdate();
20                try (ResultSet keys = ps.getGeneratedKeys()) {
21                    artistId = keys.getInt(1);
22                }
23            }
24            insertDisciplines(conn, artistId, artist.getDisciplines());
25            conn.commit();
26        } catch (SQLException e) {
27            conn.rollback();
28            throw e;
29        } finally {
30            conn.setAutoCommit(true);
31        }
32    } catch (SQLException e) {
33        throw new RuntimeException("Error saving artist", e);
34    }
35 }

```

Listing 5.2: `JdbcArtistDao.save()` — transactional insert

5.5 Connection Pool — HikariCP

The `ConnectionManager` class uses a **HikariCP** connection pool instead of opening a raw JDBC connection on every query call:

```

1 public class ConnectionManager {
2
3     private static final HikariDataSource dataSource;
4
5     static {
6         HikariConfig config = new HikariConfig();
7         config.setJdbcUrl(DatabaseConfig.URL);
8         config.setUsername(DatabaseConfig.USER);
9         config.setPassword(DatabaseConfig.PASSWORD);
10        config.setMaximumPoolSize(10);

```

```

11     config.setMinimumIdle(2);
12     config.setConnectionTimeout(30_000);
13     config.setIdleTimeout(600_000);
14     config.setMaxLifetime(1_800_000);
15     dataSource = new HikariDataSource(config);
16 }
17
18 public static Connection getConnection() throws SQLException {
19     return dataSource.getConnection();
20 }
21 }

```

Listing 5.3: ConnectionManager with HikariCP

5.6 Service Layer and Dependency Injection

The `ServiceProvider` class acts as a **manual dependency injector** (singleton factory), wiring JDBC DAOs into JDBC Services:

```

1 public class ServiceProvider {
2     private static final JdbcArtistDao artistDao = new JdbcArtistDao();
3     private static final ArtistService artistService =
4         new JdbcArtistService(artistDao);
5     // ... same pattern for all other services
6
7     public static ArtistService getArtistService() { return
8         artistService; }
9 }

```

Listing 5.4: ServiceProvider — manual DI factory (simplified)

The `InMemory` service implementations were kept in the project for offline testing but are no longer used in production.

5.7 Application Screenshots

Name	City	Email	Birth Year
Elena Moreau	Paris	elena.moreau@gmail.com	1985
Marco Bellini	Rome	marco.bellini@gmail.com	1978
Yuki Tanaka	Tokyo	yuki.tanaka@gmail.com	1992
Amara Diello	Dakar	amara.diello@gmail.com	1990
Lucas Engström	Stockholm	lucas.engstrom@gmail.com	1982
Sophie Dubois	Brussels	sophie.dubois@gmail.com	1975

(a) Artists tab

Title	Artist	Type	Price	Status
Survivor over Montmartre	Elena Moreau	Painting	4500.0	FOR_SALE
Lavender Fields Forever	Elena Moreau	Painting	3200.0	SOLD
Urban Fragments #1	Elena Moreau	Mixed Media	2800.0	EXHIBITED
The Thinker Reimagined	Marco Bellini	Sculpture	15000.0	EXHIBITED
Harmony in Stone	Marco Bellini	Sculpture	8500.0	FOR_SALE
Iron Works	Marco Bellini	Sculpture	22000.0	FOR_SALE
Neon Dreams: Shibuya	Yuki Tanaka	Photography	1800.0	FOR_SALE
Digital Garden #3	Yuki Tanaka	Digital Art	2200.0	SOLD
Tokyo Skyline	Yuki Tanaka	Photography	1200.0	FOR_SALE
Roots and Routes	Amara Diello	Mixed Media	5500.0	EXHIBITED
Market Day in Dakar	Amara Diello	Painting	3800.0	FOR_SALE
Woven Memories	Amara Diello	Mixed Media	2400.0	SOLD
Composition in Blue #12	Lucas Engström	Painting	1900.0	FOR_SALE
Blank Grid	Lucas Engström	Painting	2500.0	FOR_SALE
Ghost in the Mist	Sophie Dubois	Photography	950.0	SOLD

(b) Artworks tab

Figure 5.1: Application screens — Artists and Artworks

Title	Gallery	Start Date	Theme
Visions of Tomorrow	Galerie Lumière	2024-09-01	Futurism & Innovation
Urban Pulse	Galerie Lumière	2025-06-01	Urban Life
Sculpting the Modern World	Spazio Arte Moderna	2024-10-15	Modern Sculpture
Light & Shadow: Nordic Perspectives	Nordic Art House	2025-01-10	Nordic Minimalism
Global Threads	Gallery Horizons	2025-02-01	Cultural Identity

(a) Exhibitions tab

Name	Address	City
Galerie Lumière	15 Rue de Valenciennes, Paris 12, France	Paris
Spazio Arte Moderna	Via del Corso 18, 00186 Rome, Italy	Rome
Nordic Art House	Drottninggatan 45, 111 21 Stockholm, Sweden	Stockholm
Gallery Horizons	22 King Street, Covent Garden, London WC2E 8JQ, UK	London

(b) Galleries tab

Figure 5.2: Application screens — Exhibitions and Galleries

Title	Instructor	Date	Price	Level
Introduction to Oil Painting	Elena Moreau	2025-03-15T11:00	75.0	Beginner
Abstract Sculpture Workshop	Marco Bellini	2025-04-10T16:00	120.0	Intermediate
Night Photography Masterclass	Yuki Tanaka	2025-05-20T21:00	60.0	Intermediate
Mixed Media Collage Intensive	Ariana Diallo	2025-06-03T11:00	95.0	Advanced
Geometric Abstraction Theory & P.	Luka Engström	2025-07-12T10:00	45.0	Beginner
Advanced Color Theory for Artists	Elena Moreau	2025-08-22T15:00	85.0	Advanced

(a) Workshops tab

Name	Email	City
Alice Martin	alice.martin@email.com	Paris
Ben Thompson	ben.thompson@email.com	London
Carla Ruiz	carla.ruiz@email.com	Madrid
David Chen	david.chen@email.com	Stockholm
Elena Johansson	elena.johansson@email.com	Stockholm
Felix Nilsson	felix.nilsson@email.com	Oslo
Guillaume Petit	guillaume.petit@email.com	Lyon
Hannah Schmidt	hannah.schmidt@email.com	Berlin
Isa Fontana	isa.fontana@email.com	Marseille

(b) Community tab

Figure 5.3: Application screens — Workshops and Community

Featured Exhibitions & Content			
FEATURED EXHIBITION Visions of Tomorrow Theme: Futurism & Innovation Gallery: Galerie Lumière	FEATURED EXHIBITION Urban Pulse Theme: Urban Life Gallery: Galerie Lumière	FEATURED EXHIBITION Sculpting the Modern World Theme: Modern Sculpture Gallery: Spazio Arte Moderna	UPCOMING WORKSHOP Introduction to Oil Painting Instructor: Elena Moreau Price: \$75.0
UPCOMING WORKSHOP Abstract Sculpture Workshop Instructor: Marco Bellini Price: \$120.0	UPCOMING WORKSHOP Night Photography Masterclass Instructor: Yuki Tanaka Price: \$60.0		

Figure 5.4: Discover tab — Featured exhibitions and upcoming workshops

Chapter 6

Step 5 — Finalization and Testing

6.1 Test Scenarios

All major scenarios were tested to verify database–application consistency:

Scenario	Expected Result	Status
<i>Artist CRUD</i>		
Load all artists	Populated from <code>artist</code> + <code>artist_discipline</code> tables	
Create with 2 disciplines	Transactional INSERT into <code>artist</code> + junction table	
Update city	UPDATE + DELETE/re-INSERT disciplines (transactional)	
Delete artist	CASCADE removes artworks and disciplines	
<i>Workshop & Booking</i>		
Book a workshop	INSERT into <code>booking</code> , trigger checks capacity	
Overbook attempt	<code>trg_check_workshop_capacity</code> blocks the INSERT	
<i>Transactions</i>		
Atomic registration	All inserts succeed → COMMIT	
Failed registration	Trigger fires → ROLLBACK, member not created	
<i>Audit</i>		
Artwork status change	<code>trg_audit_artwork_status</code> inserts row in <code>audit_log</code>	

Table 6.1: Test scenarios and results

6.2 Challenges and Solutions

Challenge	Solution
No IDs in Java model classes	Surrogate INT AUTO_INCREMENT PKs in DB only. DAOs resolve IDs via unique fields (name, email).
N:M relationships in Java are simple Lists	Junction tables in DB. DAOs use batch inserts/deletes within a transaction.
Object reconstruction from flat SQL rows	Each DAO has a <code>mapRow(ResultSet)</code> helper. Children loaded with a separate batch query matched by ID.
One new connection per query	Replaced <code>DriverManager</code> with HikariCP connection pool (2–10 connections).
<code>Date.valueOf(null)</code> NPE in <code>ExhibitionDao</code>	Added null guard: <code>date != null ? Date.valueOf(date) : null</code> in <code>save()</code> and <code>update()</code> .
NPE when workshop instructor is null	Added <code>IllegalArgumentException</code> guard before opening the JDBC connection.

Table 6.2: Challenges encountered and solutions applied

Chapter 7

Conclusion

This project covered the full lifecycle of a relational database integrated into a Java application:

1. **Design** — 10 entities modeled, normalized to 3NF, 14 tables created with proper PKs, FKs, UNIQUE, and CHECK constraints.
2. **Advanced SQL** — 5 views, 6 indexes, 4 triggers, 5 stored programs, and 2 transactional scenarios exceeding the minimum requirements.
3. **Java integration** — Full 3-tier architecture (UI → Service → DAO → MySQL) with PreparedStatements, transaction handling, HikariCP connection pooling, and complete CRUD on all 6 entities.
4. **Testing** — All main scenarios verified and documented.

The main challenge was bridging the gap between the OOP model (no IDs in Java classes, object references) and the relational model (surrogate keys, junction tables). This was solved through careful DAO design that resolves IDs via unique attributes and handles multi-table writes atomically.

Project Resources

- **GitHub:** Complete source code and SQL scripts
- **SQL Scripts:** `artconnect_schema.sql`, `artconnect_data.sql`,
`artconnect_views_indexes.sql`, `artconnect_triggers.sql`,
`artconnect_procedures.sql`, `artconnect_transactions.sql`
- **Java Package:** `com.project.artconnect`
- **Database:** MySQL 8 — `artconnect`